

Deploying Software Team Analytics in a Multinational Organization

Vinay Augustine, John Hudepohl, Przemyslaw Marcinczak, and Will Snipes, ABB

// Using key indicators focusing on development team improvement, ABB found that teams could manage their activities with metrics such as cycle time. Key lessons learned include paying attention to visual design and navigation and providing drill-down capabilities for the user. //



ANALYTICS ALLOW SOFTWARE development teams to take a data-driven approach to self-improvement.

As part of its Software Development Improvement Program, ABB decided to use Qlik Sense, a business intelligence (BI) tool, to supplement the data and visualizations presented by our application lifecycle management (ALM) tools. We implemented

an analytics solution called the Qlik Sense Metrics Portal (QSMP), targeted at software development.

ABB is a multinational organization with more than 130,000 employees, with operations and staff in more than 100 countries. ABB software developers work in more than 15 countries. ABB's main businesses include industrial automation,

electrification products, power grids, and robotics and motion.

In 2009, the company embarked on the SDIP to enhance the professional development of ABB software engineers so that they could create, support, and improve best-in-class software solutions for the market.¹

In this article, we describe some of the metrics the QSMP calculates and delivers for agile teams within ABB's Grid Automation business unit. For each metric, we describe the key use cases as well as how the metric is enabled or made more effective by using a BI tool for analysis.

The metrics provided by the QSMP allow individual agile teams to review historical performance, propose improvements, and gauge their efficacy.

Metrics

We developed a number of key performance indicators (KPIs) that augment the data and visualizations provided by our ALM tools. Here, we describe four of these metrics. For each metric, we discuss why it was implemented, how the end users use it, and how it is enabled and presented by the QSMP.

Work Item Count by Status

Work item count by status is intended to highlight bottlenecks within an individual sprint. It is a stacked bar chart where each bar represents an individual day in the current sprint. The bars are color coded by the number of work items in each state.

This particular graph was developed after interviewing software developers who stated that one of the barriers to completing items in a sprint was moving work items through the states after development (examples included code reviews and acceptance tests).

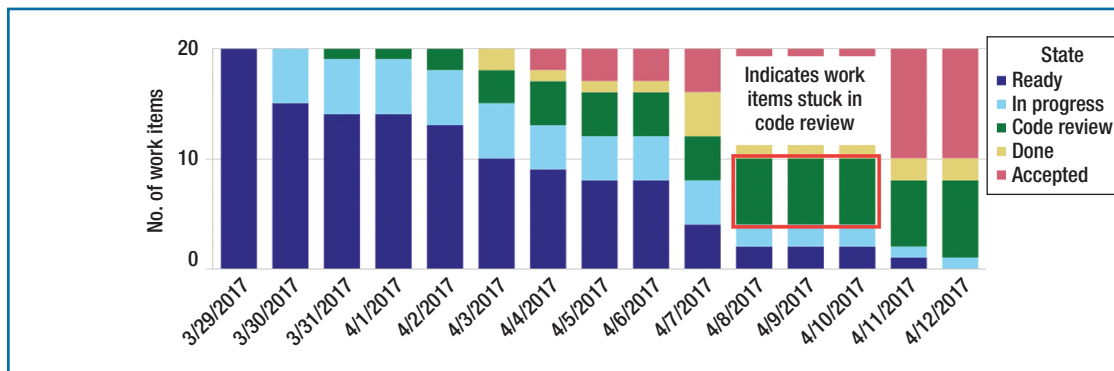


FIGURE 1. Work item count by status.

After several iterations with the developers' agile team, we created the visualization in Figure 1. This visualization lets the scrum master identify both when a team has hit a bottleneck and what state is causing the bottleneck.

With the team we interviewed, the scrum master was able to identify code reviews as a bottleneck. The team decided to ask all team members to conduct a code review in the morning before starting work. This helped resolve the bottleneck.

The raw data for this visualization was already present in our ALM tool. However, we found we could not easily format the data in order to give us this visualization. Our BI tool allowed us to easily analyze the data and create the visualization.

Cycle Time

Cycle time measures the amount of time between a work item being marked ready for development and being accepted by the business.

Cycle time primarily measures how much time the team spends working on an issue.²

By automatically computing cycle time, teams can rely on having more accurate, up-to-date data that they can use to help plan their sprints.

Teams within ABB use cycle time to help optimize their local process and to help with product release forecasts. Teams can use other analyses, such as work item count by status, to identify possible improvements in cycle time.

Prior to using the QSMP, the business unit manually collected cycle time data, aggregated it, and shared it via Excel spreadsheets. This collection was time consuming and was done only once per month. With the QSMP, cycle time is recalculated nightly and made available via a centralized dashboard.

Defect Backlog Count

The defect backlog count is measured as a 12-month rolling window of the number of opened defects waiting to be solved. Consumers of this metric can filter it by product, release, priority, and source (internal versus external). The defect backlog count is presented as a KPI and provides a high-level overview of the product's quality.

As with cycle time, the defect-backlog-count metric was previously calculated via a manually updated spreadsheet that collected and analyzed the data.

In addition to the benefits gained from automatically computing the

defect backlog counts, the QSMP enables additional views of this data.

One of the major benefits of BI solutions is that they allow users to combine disparate data sources. In this case, the business unit took the defect backlog counts and combined them with commits and file paths from the source code repository.

Figure 2 shows a merged view of two data sources for commits and defects. It allows product owners and architects to find the most defect-prone parts in the code base. Each box represents a file or higher-level path and is sized by the number of defect fixes in that path portion. The boxes are clickable so that the user can drill down through the path hierarchy to the file level to see the most defect-prone files.

Users can use this to target refactoring and other architectural improvements and then measure the efficacy of those improvements.

Code Ownership

Our final metric illustrates the usefulness of combining different data sources in a different scenario.

One of the issues we see with our global set of software developers is that they often do not know whom to contact for guidance when

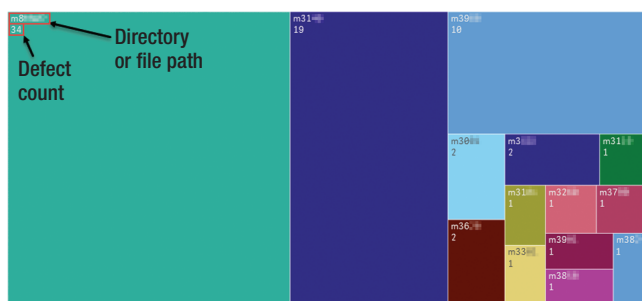


FIGURE 2. A code heat map with defects.

modifying an unfamiliar part of the project. Teams also have to worry about owners of particular parts of the code departing the company (either internally or externally).

Teams can use the code ownership map (see Figure 3) to navigate these issues. This map was inspired by the knowledge map created by Adam Tornhill.³

Figure 3 shows all of the files for a code base. This view can be filtered from the root of a repository all the way to individual files. The heat map shows each file as a rectangle. Within each file, the individual contributors are represented proportionally to the number of commits they have made to that path. This map makes it easier to identify the primary contributors for a file. While the commit history also has this information, it does not differentiate between someone who has made numerous changes to a file and someone who has made only one.

The code ownership map can also be combined with the company's internal LDAP (Lightweight Directory Access Protocol) directory. This means we can identify parts of the code base that are abandoned owing to primary contributors changing roles or departing the company.

Results

By migrating from a set of spreadsheets to a centralized business solution, ABB has gained the ability to automatically calculate and present a set of actionable metrics for its various development teams.

Prior to implementing the QSMP, executives, managers, and team members were only able to get monthly reports generated manually by a team of quality personnel. We set out to build a dashboard solution to replace the quality reports and automate the process so that the data were updated nightly.

By combining the data collected with additional sources of data (such as source code and personnel data), we were able to enable additional analytics that teams previously struggled with.

One impact of analytics was improved consistency of process implementation. While groups were following the same processes, they had not always used common terminology and measurements. The software engineering analytics solution highlighted data quality issues, incomplete data, and inaccurate data. The team was able to correct the implementations to provide consistent reporting of metrics like cycle time.

Related Work

In “Software Analytics: So What?,” the guest editors’ introduction to the July/August 2013 *IEEE Software* Special Issue on Software Analytics, Tim Menzies and Thomas Zimmermann promote the concept of user-centered analytics, whereas we focus on decision support for the development manager and project manager.⁴ They offer a set of seven principles for software analytics, the first of which is “users before algorithms,” suggesting that algorithms should support user goals.

Giulio Concas and his colleagues model a maintenance process and compare the resulting cycle time for projects with a work-in-progress limit versus projects without such a limit.⁵ They find setting a work-in-progress limit reduces cycle time.

Zhitao Hou and his colleagues develop a visual software engineering data exploration tool called MetroEyes that allows users to perform data exploration visually, by clicking on objects in charts and graphs.⁶

Irina Corman and her colleagues discuss a case study of deploying an automated software engineering measurement system that uses modern tools to instrument the developer’s activity in the IDE.⁷ They learned that developer buy-in is critical, measurement programs require a long runway to reach deployment, and action-oriented presentation of measurements is vitally important.

Ryan Polk describes a case study implementing a Kanban methodology for software maintenance activities and defines key metrics used to evaluate the effectiveness of the Kanban team.⁸ He defines team cycle time as the average cycle time per story point and cycle time as the time required to complete a story of a given size.

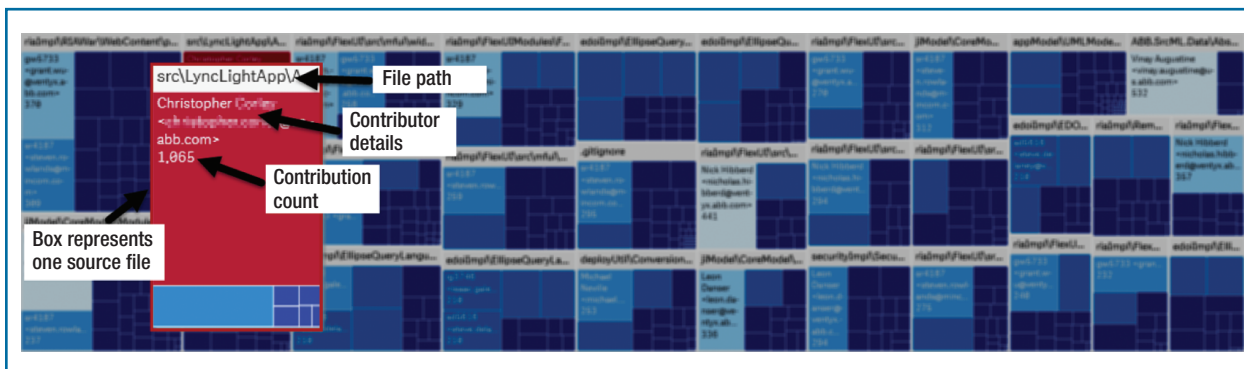


FIGURE 3. A code ownership heat map (blurred to obscure the names).

Several works discuss methods for and lessons learned from implementing a corporate metrics program.^{9–12} Key lessons learned include decentralizing the selection of metrics, involving developers in the design stage, frequent metric data collection, and involving management stakeholders in goal setting. In our program, we followed many of these guidelines. Although we chose to centralize the analytics system, we applied it using a tailorable framework allowing groups to create their own metrics dashboards.

Mirosław Staron and Wilhelm Meding define metrics based on a process flow model of an agile process operating in an industrial environment.¹³ They define the throughput metric as the number of elements processed during a time period. We measure throughput as the number of stories completed during each sprint.

ABB built a software engineering analytics solution reflecting the best ways to measure current practices. There were several lessons learned during the rollout of the QSMP related to stakeholder interaction and design considerations.

When interacting with stakeholders, we learned to engage business units with metric success stories. To encourage usage of the analytics portal, we track how much and when people in each business unit are using it.

When designing dashboards, we incorporated features that provide fast visualization rendering and filtering, increase interactivity, and improve ease of navigation. This is the key to overcoming the drawbacks with other metric delivery solutions. Another key aspect of design was implementing a drill-down capability allowing the user to dig into details behind a high-level chart.

The analytics solution improved process adherence as metrics were rolled out to multiple teams. Scrum teams were able to monitor and reduce key indicators such as cycle time. 

Acknowledgments

We thank our ABB colleagues, especially Utpal Gandhi, Scott Madden, and Patrick Ryan, for their support and active participation in this venture.

References

1. J. Hudepohl et al., “Deploying an Online Software Engineering

Education Program in a Globally Distributed Organization,” *Companion Proc. 36th Int’l Conf. Software Eng. (ICSE 14)*, 2014, pp. 301–310.

2. V.R. Basili et al., “The Goal Question Metric Approach,” *Encyclopedia of Software Engineering*, vol. 2, 1994, pp. 528–532.
3. A. Tornhill, *Your Code as a Crime Scene: Use Forensic Techniques to Arrest Defects, Bottlenecks, and Bad Design in Your Programs*, Pragmatic Bookshelf, 2015.
4. T. Menzies and T. Zimmermann, “Software Analytics: So What?,” *IEEE Software*, vol. 30, no. 4, 2013, pp. 31–37.
5. G. Concas et al., “Simulation of Software Maintenance Process, with and without a Work-in-Process Limit,” *J. Software: Evolution and Process*, vol. 25, no. 12, 2013, pp. 1225–1248.
6. Z. Hou et al., “Visual Analytics for Software Engineering Data,” *Perspectives on Data Science for Software Engineering*, Morgan Kaufmann, 2016.
7. I.D. Coman, A. Sillitti, and G. Succi, “A Case-Study on Using an Automated In-Process Software Engineering Measurement and Analysis System in an Industrial Environment,” *Proc. 31st Int’l Conf. Software Eng. (ICSE 09)*, 2009, pp. 89–99.



VINAY AUGUSTINE is a lead principal scientist at ABB Corporate Research. His research interests include developer and team productivity, agile software development, and open source ecosystems. Augustine received his PhD in computer science from Case Western Reserve University. Contact him at vinay.augustine@us.abb.com.



JOHN HUDEPOHL is a project manager in the ABB R&D Lean Product Development organization and the product owner of the Qlik Sense Metrics Portal for ABB R&D. His interests include developing and deploying solutions for ABB R&D, including analytic dashboards for relevant individual developers, development teams, and senior management. John received an MS in systems management from the Florida Institute of Technology. Contact him at john.hudepohl@us.abb.com.



PRZEMYSŁAW MARCINCZAK is a business intelligence information systems application manager at ABB Business Services. His interests include business intelligence, big data, software architecture, data modeling, software metrics, project management metrics, and financial metrics. Przemyslaw received an MS in finance and banking from the University of Lodz. Contact him at przemyslaw.marcinczak@gmail.com.



WILL SNIPES is a principal scientist at ABB Corporate Research. His research interests include influencing practice by providing analytics relevant to individual developers and development teams. Snipes received an MS in computer science from North Carolina State University. Contact him at will.snipes@us.abb.com.

Hawaii Int'l Conf. System Sciences (HICSS 00), 2000.

13. M. Staron and W. Meding, "Monitoring Bottlenecks in Agile and Lean Software Development Projects—a Method and Its Industrial Use," *Product-Focused Software Process Improvement*, Springer, 2011, pp. 3–16.

8. R. Polk, "Agile and Kanban in Coordination," *Proc. 2011 Agile Conf. (AGILE 11)*, 2011, pp. 263–268.
9. S.L. Pfleeger, "Lessons Learned in Building a Corporate Metrics Program," *IEEE Software*, vol. 10, no. 3, 1993, pp. 67–74.
10. M.K. Daskalantonakis, "A Practical View of Software Measurement and Implementation Experiences within

Motorola," *IEEE Trans. Software Eng.*, vol. 18, no. 11, 1992, pp. 998–1010.

11. T. Hall and N. Fenton, "Implementing Effective Software Metrics Programs," *IEEE Software*, vol. 14, no. 2, 1997, pp. 55–65.
12. J. Iversen and L. Mathiassen, "Lessons from Implementing a Software Metrics Program," *Proc. 33rd Ann.*

myCS

Read your subscriptions through the myCS publications portal at

<http://mycs.computer.org>